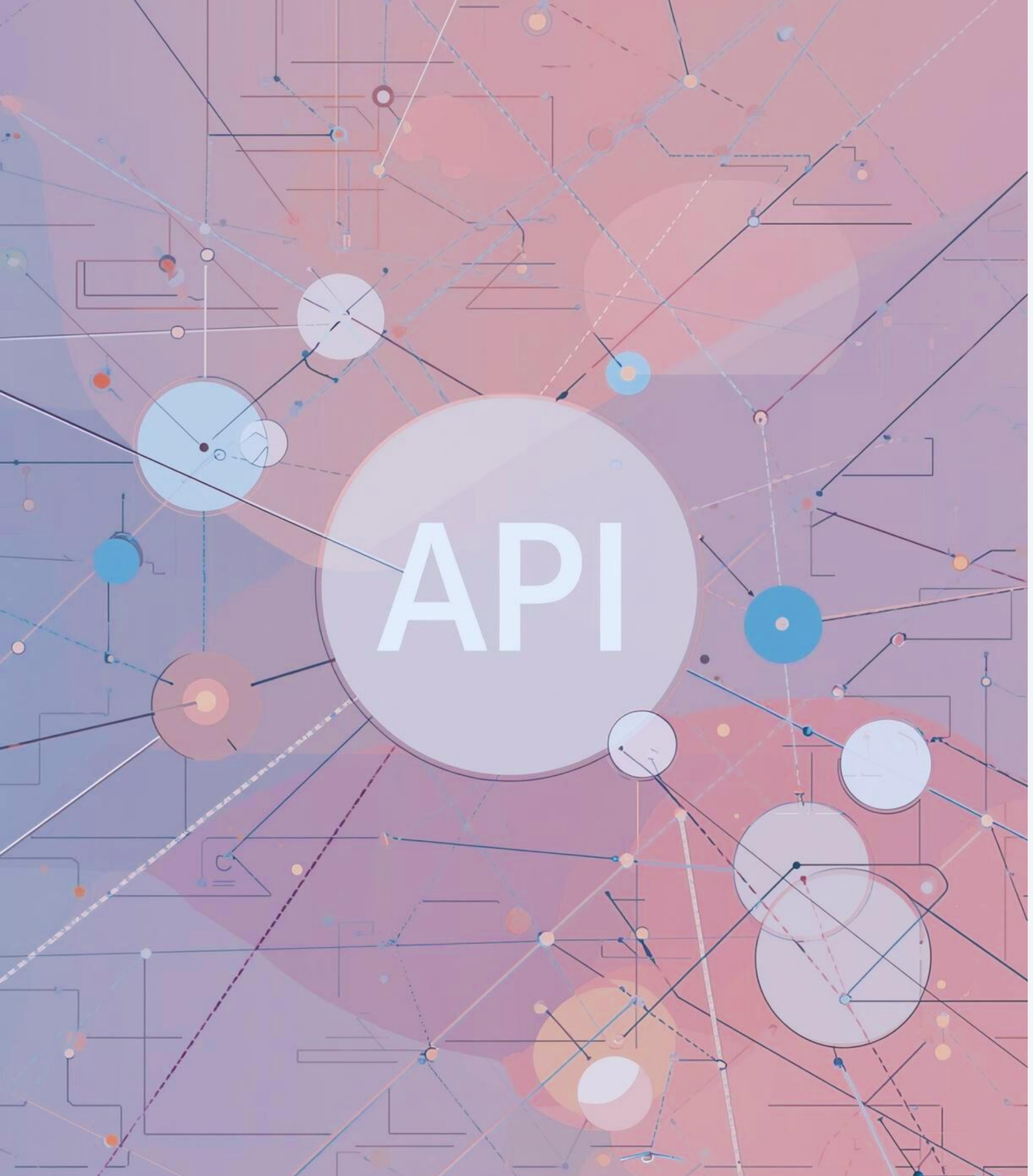
DOCENTE	**Fabio ABBA’**
UNITA’ FORMATIVA	**Competenze informatiche/digitali**
ARGOMENTO	**RESTful API**



OpenAPI Solutions

Introduzione a OpenAPI

Understanding API Standards

PRESENTED BY

Fabio ABBA'



OpenAPI

OpenAPI (precedentemente Swagger Specification) è uno standard industriale per descrivere API REST in formato machine-readable (tipicamente JSON o YAML).

Definisce un formato strutturato che descrive tutti gli endpoint della nostra API, i parametri che accettano, gli schemi dei dati in input e output, i codici di stato HTTP possibili, i metodi di autenticazione, esempi, descrizioni testuali.

Ecosistema Swagger

SWAGGER È L'ECOSISTEMA DI STRUMENTI CHE IMPLEMENTA QUESTA SPECIFICA, COMPRENDENDO EDITOR, GENERATORI DI CODICE E INTERFACCE INTERATTIVE.

SWAGGER EDITOR

Swagger Editor è uno strumento online che permette di scrivere e modificare specifiche OpenAPI in modo (quasi) intuitivo.

SWAGGER UI

Swagger UI fornisce un'interfaccia web interattiva per visualizzare e testare le API.

SWAGGER CODEGEN

Swagger Codegen è un generatore automatico di client e server che produce codice sorgente in vari linguaggi. Accelera lo sviluppo e garantisce coerenza tra implementazione e documentazione API.

Vantaggi della documentazione OpenAPI

STANDARDIZZAZIONE

L'uso di OpenAPI garantisce un formato **universalmente riconosciuto**, facilitando la comunicazione e l'integrazione tra team diversi e sistemi.

AUTOMAZIONE

Grazie a strumenti come Swagger, è possibile **ridurre errori manuali** e mantenere la sincronizzazione tra codice e documentazione, generando automaticamente il codice necessario.

TESTING INTEGRATO

La documentazione OpenAPI consente di **testare direttamente gli endpoint** dall'interfaccia, rendendo il processo di verifica più semplice e immediato per gli sviluppatori.

Tools aggiuntivi

Vediamo subito in pratica quanto detto.
Per documentare le API della nostra applicazione avremo bisogno di un tool aggiuntivo che ci semplifichino il lavoro:

- **swagger-ui-express**: genererà l'interfaccia a partire da un file di configurazione (JSON in questo caso, ma più comunemente YAML)



Riferimento:
https://medium.com/@im_AnkitTiwari/swaggerizing-your-node-js-rest-api-a-step-by-step-guide-267255bf8bbe

Installazione

Cominciamo ad installare quanto serve:

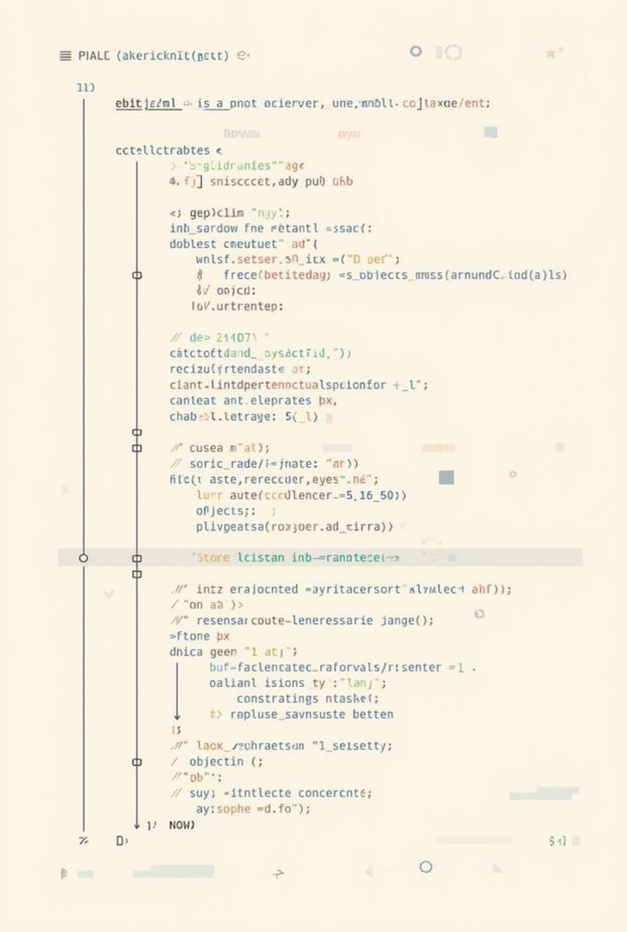
```
=> npm install swagger-ui-express

=> npm install swagger-autogen
```



E creiamo (se non l'abbiamo già fatto) una directory "doc“ (o swagger) dove inserire il file che servirà a generare l’interfaccia.

Esempio JSON



Questo è un esempio in formato JSON.


```
  "description": "Progetto di esempio per una libreria online",
  "version": "1.0.0"
},
"host": "localhost:3000",
"basePath": "/api/v1",
"tags": [
  {
    "name": "Catalogo",
    "description": "Operazioni relative al catalogo dei libri"
  },
  {
    "name": "Clienti",
    "description": "Operazioni relative ai clienti della libreria
  },
  {
    "name": "Ordini",
    "description": "Operazioni relative agli ordini dei clienti"
  }
],
"schemes": [
  "http"
],
"consumes": [
  "application/json"
],
"produces": [
  "application/json"
],
"paths": {
  "/catalogo/": {
    "get": {
      "tags": [
        "Catalogo"
      ],
      "description": "endpoint per recuperare l'intero catalogo",
```

Esempio YAML



Questo è un esempio in formato YAML.
E' possibile dare il file .yml in pasto a dei servizi che sulla base di questo contenuto creeranno automaticamente la struttura documentata delle API che esponiamo.
Molto però dipende dal tipo di architettura usata.

```yaml
basePath: /api/v1
tags:
  - name: Catalogo
    description: Operazioni relative al catalogo dei libri
  - name: Clienti
    description: Operazioni relative ai clienti della libreria
  - name: Ordini
    description: Operazioni relative agli ordini dei clienti
schemes:
  - http
consumes:
  - application/json
produces:
  - application/json
paths:
  /catalogo/:
    get:
      tags:
        - Catalogo
      description: endpoint per recuperare l'intero catalogo
      responses:
        '200':
          description: OK
    post:
      tags:
        - Catalogo
      description: endpoint per aggiungere un nuovo libro
      parameters:
        - name: body
          in: body
          schema:
            type: object
            properties:
              id:
                example: any
      responses:
        '201':
          description: Created
```

Generazione interfaccia

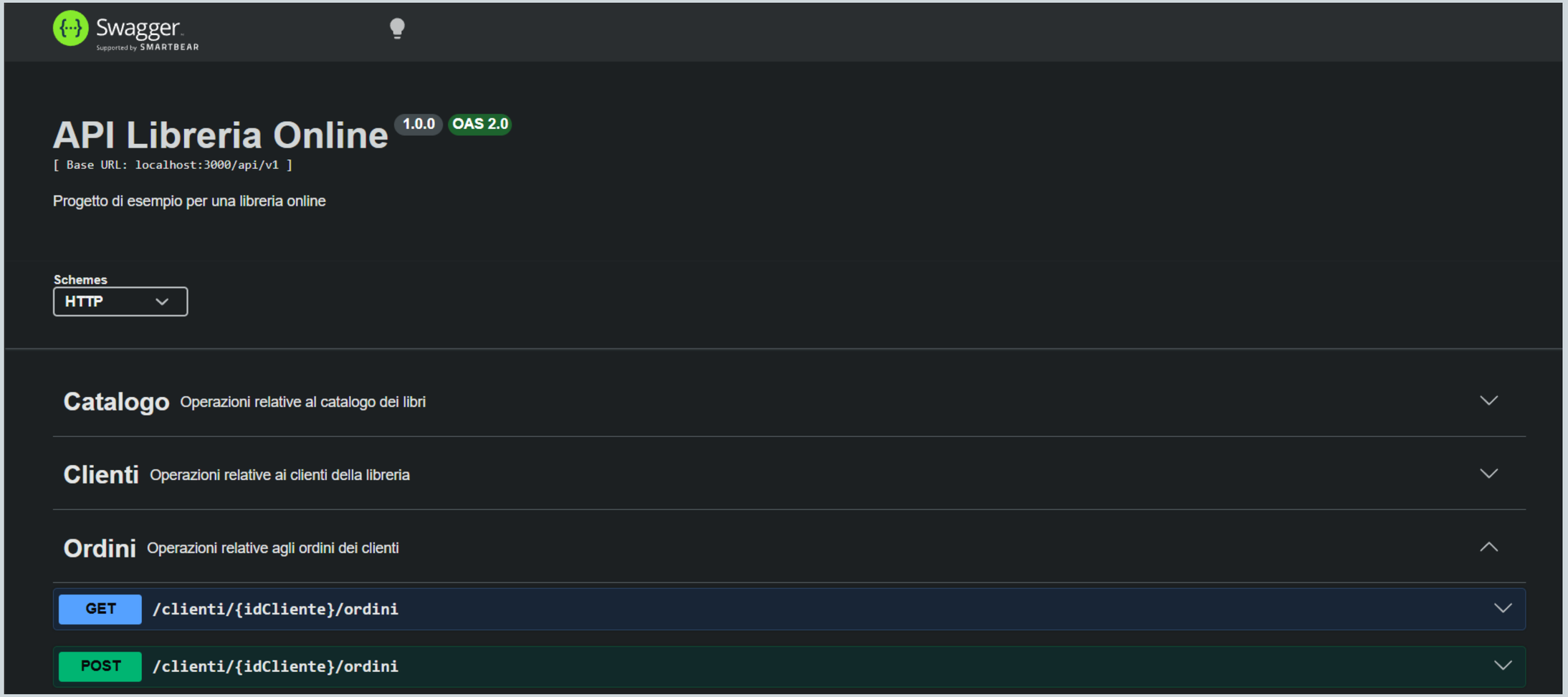
Per quanto ci riguarda, noi useremo il file JSON per generare l'interfaccia Swagger.

Aggiungiamo il codice necessario:

```
5   import swaggerUI from 'swagger-ui-express';
6   import swaggerjsonFilePath from './doc/swagger-output.json' with { type: "json" };
7
8   const app = express();
9   app.use(bodyParser.json());
10  const port = process.env.PORT || 3000;
11  const BASE_PATH = '/api/v1';
12
13
14  /*
15  Per mantenere più pulito il codice, c'è la possibilità di utilizzare
16  gli "Express Router" per suddividere le varie parti dell'API in moduli separati.
17  */
18  app.use(`${BASE_PATH}/catalogo`, catalogoRouter);
19  app.use(`${BASE_PATH}/clienti`, clientiRouter);
20
21  // integrazione di Swagger UI per la documentazione interattiva dell'API
22  app.use("/api-docs", swaggerUI.serve, swaggerUI.setup(swaggerjsonFilePath));
```

Interfaccia Swagger

Ora, se avviamo l'applicazione e andiamo sulla pagina "**/api-docs**", avremo a disposizione un'interfaccia che ci mostra gli endpoint disponibili e ci permette di testarli.

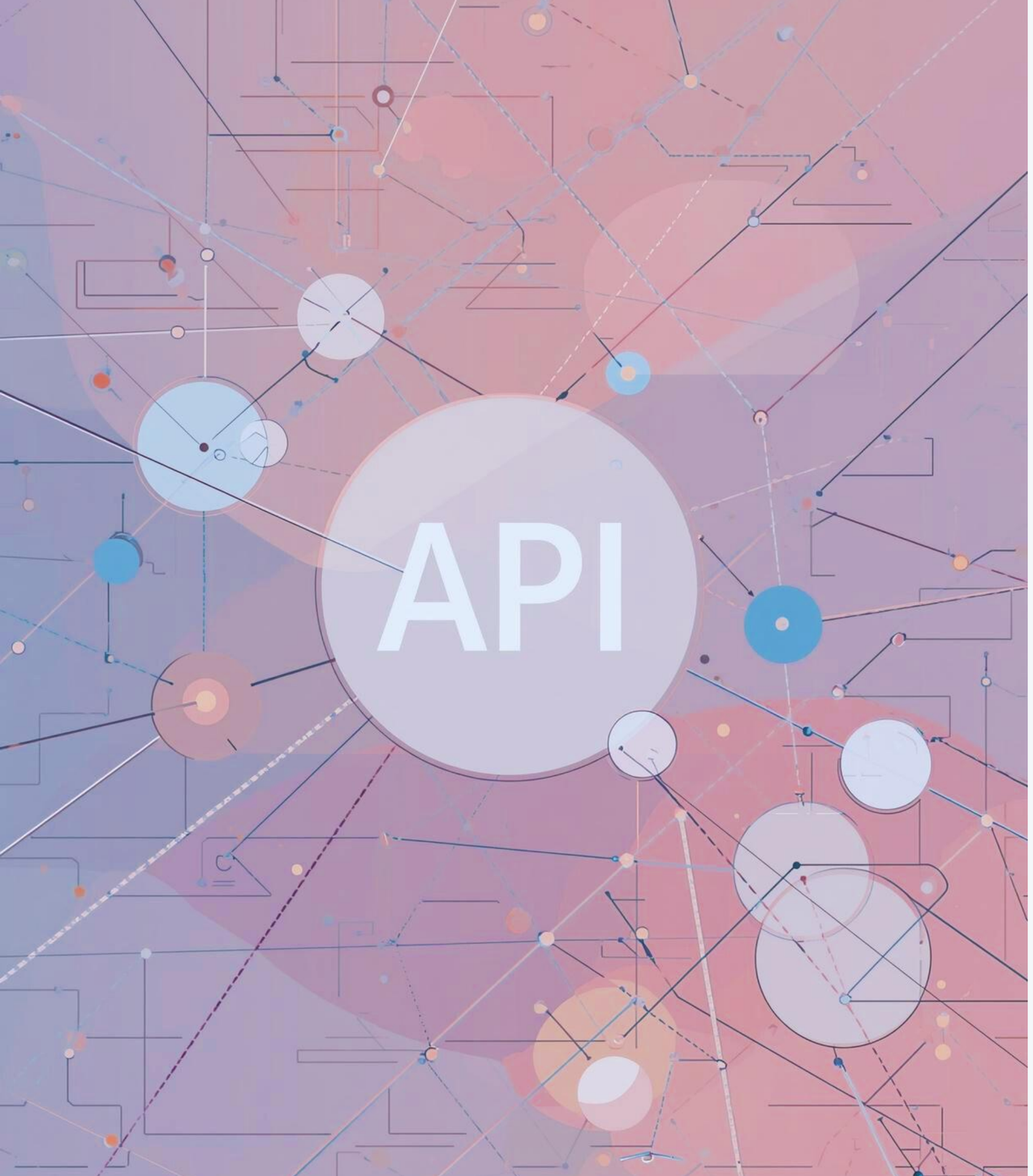


Oltre all'interfaccia intuitiva, c'è un altro grande vantaggio in questo approccio: una volta definita la struttura delle API nel file **swagger.yml** (e sperando che non cambi nel frattempo!), esso fungerà da riferimento per gli sviluppi sia lato frontend che lato backend anche in assenza delle implementazioni complete e in maniera completamente indipendente.

Risorse utili

- https://swagger.io/docs/specification/v3_0/about/
- https://editor.swagger.io
- https://medium.com/@im_AnkitTiwari/swaggerizing-your-node-js-rest-api-a-step-by-step-guide-267255bf8bbe
- https://swagger-autogen.github.io/docs/





COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Grazie per l'attenzione!

PRESENTED BY

Fabio ABBA'

fabio.abba@tc-web.it

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

TC-Web Technology Consulting

Via Giovanni Schiaparelli 14,
10148 – Torino (TO)
E-mail: info@tc-web.it
Tel: 011 0466685